Ex.No: 07	PL / SQL - TRIGGERS

Objective:

To solve the given problems using triggers.

Description:

Triggers are similar to stored procedures. A trigger stored in the database can include SQL and PL/SQL or Java statements to run as a unit and can invoke stored procedures. However, procedures and triggers differ in the way that they are invoked. A procedure is explicitly run by a user, application, or trigger.

Detailed Procedure:

The events that fire a trigger include the following:

DML statements that modify data in a table (INSERT, UPDATE, or DELETE) DDL statements System events such as startup, shutdown, and error messages User events such as logon and logoff A trigger has three basic parts:A triggering event or statement ,trigger restriction , trigger has three basic parts:riggering event or statement ,trigger restriction A trigger action

The types of triggers are:

BEFORE and AFTER Triggers

INSTEAD OF Triggers

Triggers on System Events and User Events

A database trigger is procedural code that is automatically executed in response to certain events on a particular table or view in a database. The trigger is mostly used for keeping the integrity of the information on the database. For example, when a new record (representing a new worker) is added to the employees table, new records should be created also in the tables of the taxes, vacations, and salaries.

Sample Input /Output

1. Create a trigger T1_sal that prints “Salary incremented “whenever there is a increase in salary and “salary decremented “whenever there is a decrease in salary in employees table

```
create or replace trigger t1_sal
after update of salary on employee
```

```
for each row
Begin
 if (:new.salary>:old.salary) then
     dbms_output.put_line('Salary Incremented');
 elsif (:new.salary<:old.salary) then
 dbms_output.put_line('Salary Decremented');
 end if;
 end;
```

2. Create a trigger T2 _error that raises an error whenever there is no commission for manager for new insertions

```
create or replace trigger t2_error
    after insert on emp
    for each row
    begin
    if(:new.comm is null) then
    raise_application_error(-20000,'commission not given');
    end if;
    end;
```

3. Create a trigger T3 _set_null to set the manager id null in the departments table whenever the manager id is deleted from the employees table.

```
 create or replace trigger t3_set_null
     after delete  on employee
     for each row
     begin
     update departments set manager_id=null where
manager_id=:old.manager_id;
     end;
```

Questions: -

1. Whenever there is a new conference event is created, make sure it is conducted in venue id 5.

2. Create a trigger to implement referential integrity policy – “on delete set null’ in the event table. That is when venue id is deleted from the venue table, set null for deleted venue in the event table.

3. Create a trigger that prints the change in venue every time venue of an event is changed.

4. Write a trigger to stop the events happening in venue “ Grand Theater“.

5. Create a trigger for ticket table that inserts the newly inserted values in the ticket_audit table in case of insert and inserts old existing values in ticket_audit table in case of an change on status ‘confirmed’ from ‘reserved’. The structure of the ticket_audit table is same as ticket table.

6. Disable the above created trigger.

7. Drop the first created trigger.